

SmartClinic

Database Management System

Relational Database Design & Implementation in MySQL

Project Documentation

Entity-Relationship Design • Relational Mapping • Use Case Modeling

Schema, Triggers, Views, Stored Procedures & Indexing

Table of Contents

1. Project Overview	3
2. Entity-Relationship Diagram (ERD)	4
3. Relational Mapping	6
4. Use Case Diagram	7
5. Database Schema	9
5.1 DOCTORS	9
5.2 PATIENTS	9
5.3 APPOINTMENTS	10
5.4 BILLS	10
5.5 PRESCRIPTIONS	11
5.6 ROOMS	11
5.7 Patient_Doctor_Consultation	11
6. Triggers.....	12
7. Views.....	15
8. Stored Procedures	16
9. Indexing.....	18
10. Subqueries	19
11. Project Summary	21

1. Project Overview

SmartClinic is a relational database system designed to manage the core operations of a small-to-medium medical clinic: patient records, doctor profiles, appointment scheduling, prescriptions, billing, and room/resource management. The project was built entirely in MySQL, moving from conceptual design through to a working, rule-enforcing implementation.

The design process followed a standard database engineering workflow: modeling entities and relationships with an Entity-Relationship Diagram (ERD), translating that model into a relational schema (Mapping), defining system behavior from a user's perspective with a Use Case Diagram, and finally implementing the schema with triggers, views, stored procedures, and indexes to enforce business rules and support real application workflows.

Objectives

- Model a realistic multi-entity healthcare domain with correct relationship cardinalities (1:1, 1:M, M:N).
- Translate the conceptual model into a normalized relational schema with appropriate keys and constraints.
- Automate business logic (billing calculations, room availability, payroll) at the database layer using triggers.
- Simplify common data access patterns with views.
- Encapsulate reusable operations (registration, discharge, payment processing) with stored procedures.
- Improve query performance on frequently filtered/joined columns with indexing.

Tech Stack

- Database Engine: MySQL
- Modeling Tools: draw.io (ERD, Relational Mapping, Use Case Diagram)
- Version Control: Git / GitHub

2. Entity-Relationship Diagram (ERD)

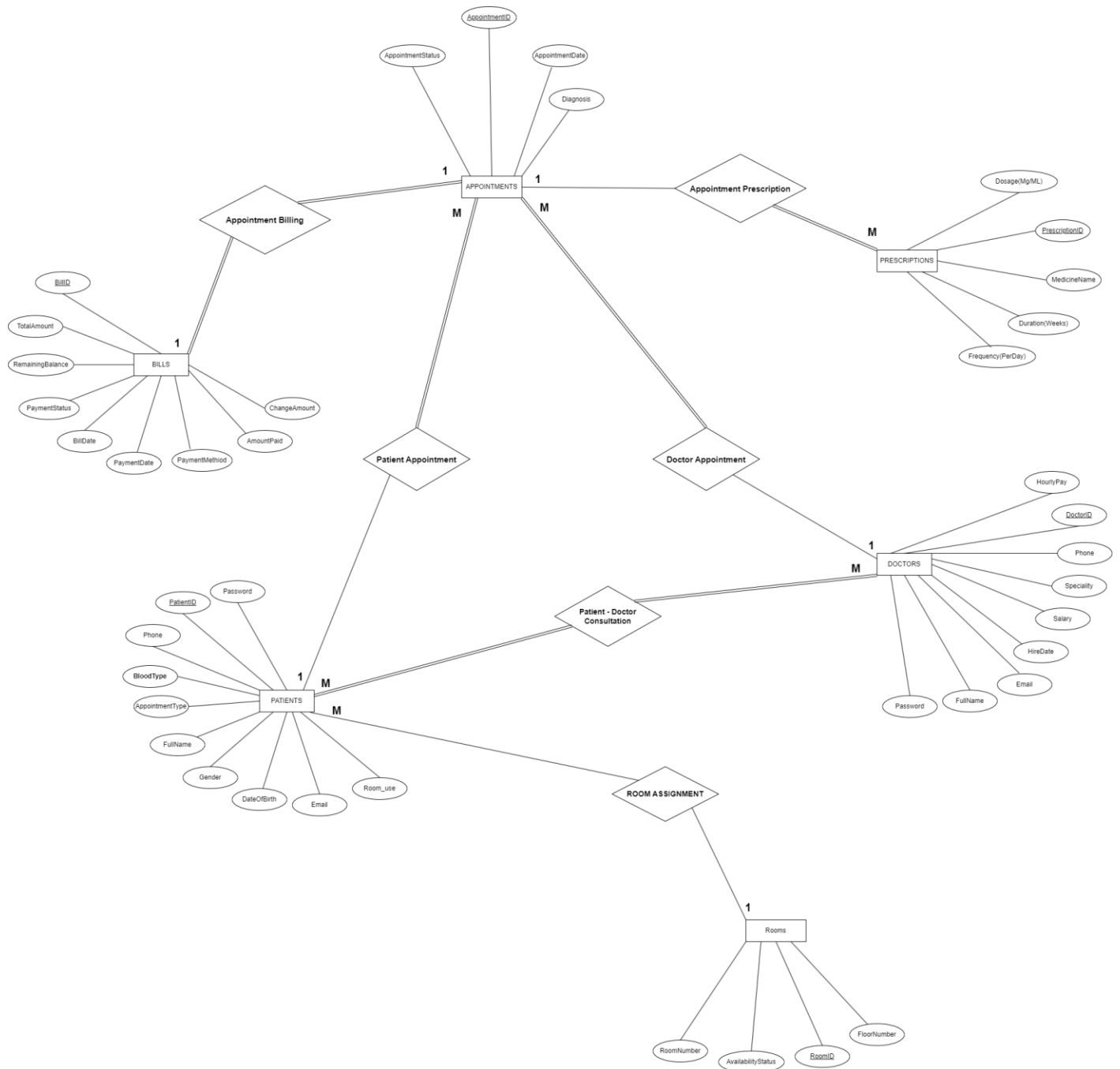


Figure 1 — SmartClinic Entity-Relationship Diagram

The ERD models the clinic's domain at a conceptual level: seven core entities and the relationships that connect them, including cardinality (1 / M) on every relationship. This diagram was the starting point for the entire schema — every foreign key in the final database traces back to a relationship defined here.

Core Entities

- DOCTORS — clinic physicians, including specialty and payroll information.
- PATIENTS — registered patients, including their assigned room (if any).
- APPOINTMENTS — the central transaction entity, linking one patient to one doctor per visit.
- PRESCRIPTIONS — medicines prescribed during a specific appointment.
- BILLS — billing and payment records tied to a specific appointment.
- ROOMS — physical hospital rooms and their occupancy status.
- Patient–Doctor Consultation — a direct many-to-many association between doctors and patients, independent of individual appointments.

Key Relationships & Cardinality

- APPOINTMENTS to BILLS — 1:M (Appointment Billing) — one appointment can generate multiple bills.
- APPOINTMENTS to PRESCRIPTIONS — 1:M (Appointment Prescription) — one appointment can produce multiple prescriptions.
- PATIENTS to APPOINTMENTS — 1:M (Patient Appointment) — a patient can have many appointments over time.
- DOCTORS to APPOINTMENTS — 1:M (Doctor Appointment) — a doctor can hold many appointments.
- DOCTORS to PATIENTS — M:N (Patient–Doctor Consultation) — modeled as its own associative entity, separate from APPOINTMENTS, to represent an ongoing consultation relationship rather than a single visit.
- PATIENTS to ROOMS — M:1 (Room Assignment) — many patients can be assigned rooms over time, but each patient occupies at most one room at once.

3. Relational Mapping

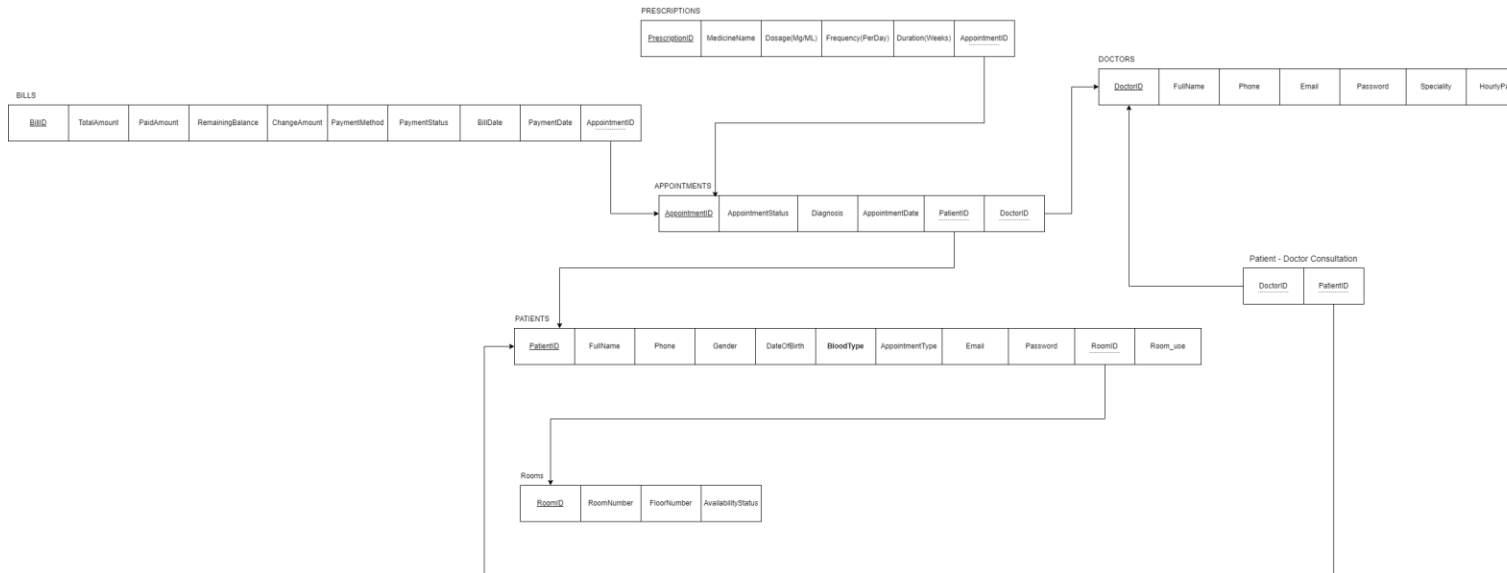


Figure 2 — Relational Mapping of the SmartClinic Schema

The Mapping diagram shows how the conceptual ERD was translated into concrete relational tables, with primary keys (underlined) and the actual foreign key columns and their direction of reference. This is the direct blueprint for the SQL CREATE TABLE statements.

Mapping Decisions Worth Noting

- Every 1:M relationship from the ERD became a foreign key placed on the "many" side (e.g. AppointmentID sits on BILLS and PRESCRIPTIONS, not on APPOINTMENTS).
- The M:N Patient–Doctor Consultation relationship became its own junction table with a composite primary key (DoctorID, PatientID), rather than a single-column surrogate key — since the combination itself is the natural identifier.
- PATIENTS.RoomID is nullable, since a patient is not required to occupy a room at time of registration.
- Column names from the original design that used disallowed MySQL characters — Dosage(Mg/ML), Frequency(PerDay), Duration(Weeks) — were normalized to Dosage_MgML, Frequency_PerDay, and Duration_Weeks.

4. Use Case Diagram

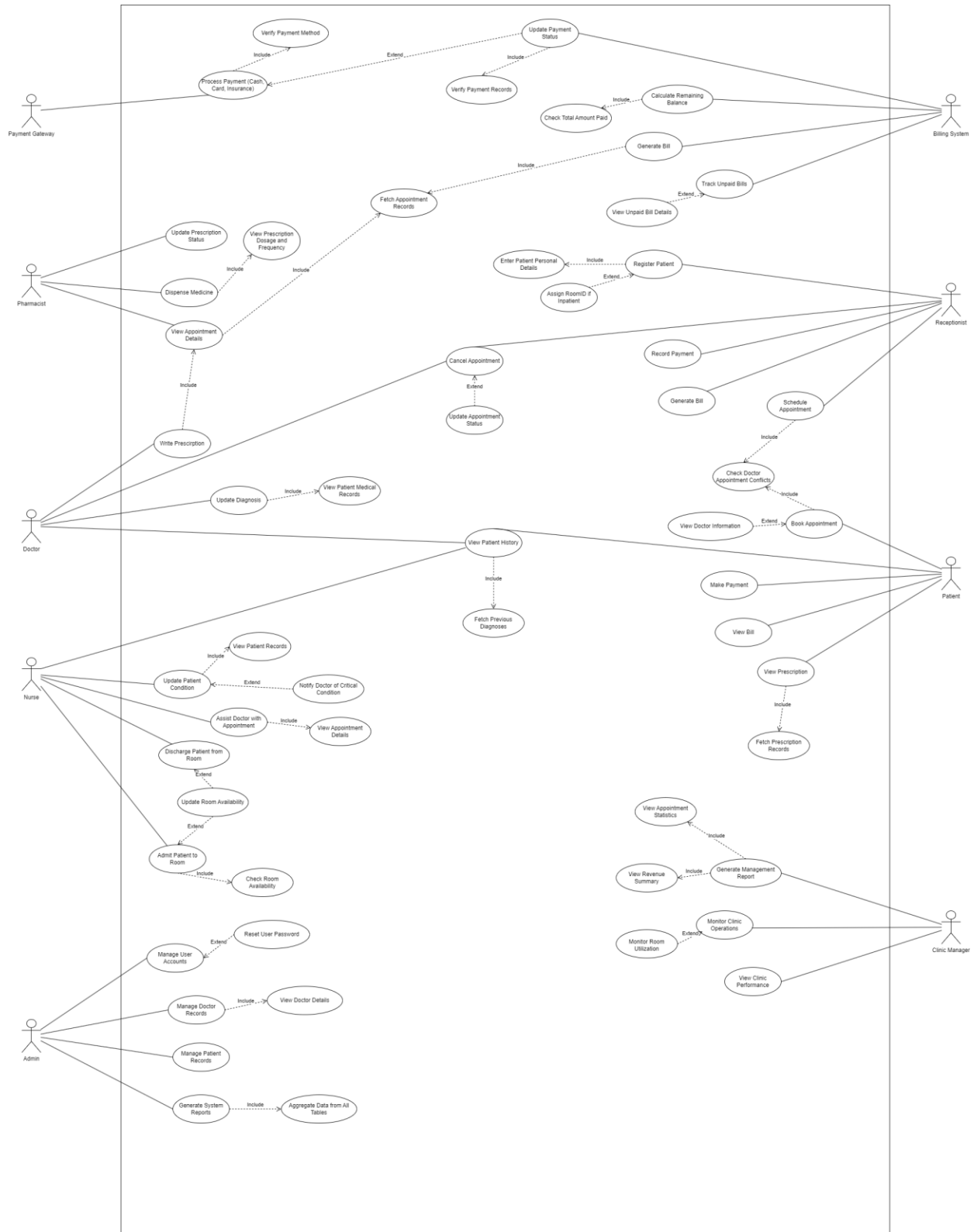


Figure 3 — SmartClinic System Use Case Diagram

The Use Case Diagram models the system from the perspective of every actor who interacts with it — not just the database, but the full clinic application built on top of it. It captures the roles (Patient, Doctor, Nurse, Receptionist, Pharmacist, Admin, Clinic Manager, and external systems like the Payment Gateway and Billing System) and the actions available to each, including include/extend relationships between related use cases.

Actors

- Patient — books/cancels appointments, views bills and prescriptions, makes payments.
- Doctor — writes prescriptions, updates diagnoses, views patient history.
- Nurse — manages patient conditions, room admission/discharge, assists doctors.
- Receptionist — registers patients, schedules appointments, records payments, generates bills.
- Pharmacist — dispenses medicine, updates prescription status.
- Admin — manages user accounts, doctor/patient records, and system-wide reports.
- Clinic Manager — monitors clinic operations, room utilization, and generates management reports.
- Billing System / Payment Gateway — external systems supporting payment verification and processing.

Design Note

Several use cases directly map to the database-layer automation implemented later in this document — for example, "Discharge Patient from Room" and "Update Room Availability" correspond to the DischargePatient stored procedure and the Room_Availability triggers; "Record Payment" and "Calculate Remaining Balance" correspond to the Insert_Bills / Update_Bills triggers and ProcessPayment procedure.

5. Database Schema

The schema consists of seven tables. Primary keys are auto-incrementing surrogate keys except where a composite natural key made more sense (Patient_Doctor_Consultation) or where the ID is manually managed (ROOMS).

5.1 DOCTORS

Stores doctor accounts, specialty, and payroll information.

Column	Type	Purpose
DoctorID	INT, PK, AUTO_INCREMENT	Unique identifier for each doctor.
FullName	VARCHAR(100)	Doctor's full name.
Phone	VARCHAR(20)	Contact number.
Email	VARCHAR(100), UNIQUE	Login identifier.
Password	VARCHAR(255)	Hashed account password.
Speciality	VARCHAR(100)	Medical specialty.
HourlyPay	DECIMAL(10,2)	Base hourly pay rate — drives the Salary calculation trigger.
Salary	DECIMAL(10,2)	Auto-calculated monthly salary.
HireDate	DATE	Date the doctor joined the clinic.

5.2 PATIENTS

Stores patient accounts and their current room assignment.

Column	Type	Purpose
PatientID	INT, PK, AUTO_INCREMENT	Unique identifier for each patient.
FullName	VARCHAR(100)	Patient's full name.
Phone	VARCHAR(20)	Contact number.
Gender	VARCHAR(10)	Patient's gender.
DateOfBirth	DATE	Used for age-related calculations.
BloodType	VARCHAR(5)	Patient's blood type.
AppointmentType	VARCHAR(50)	Preferred/default appointment type.
Email	VARCHAR(100), UNIQUE	Login identifier.
Password	VARCHAR(255)	Hashed account password.
RoomID	INT, FK → ROOMS	Currently assigned room; nullable.
Room_use	BOOL, NOT NULL	Whether the patient is actively occupying their room. Drives room-availability automation.

5.3 APPOINTMENTS

The central transactional table — links one patient with one doctor for a specific visit.

Column	Type	Purpose
AppointmentID	INT, PK, AUTO_INCREMENT	Unique identifier for each appointment.
AppointmentStatus	VARCHAR(30)	Scheduled / Completed / Cancelled.
Diagnosis	VARCHAR(255)	Doctor's diagnosis notes.
AppointmentDate	DATETIME	Date and time of the visit.
PatientID	INT, FK → PATIENTS	The patient being seen.
DoctorID	INT, FK → DOCTORS	The attending doctor.

5.4 BILLS

Billing and payment tracking, tied to a specific appointment.

Column	Type	Purpose
BillID	INT, PK, AUTO_INCREMENT	Unique identifier for each bill.
TotalAmount	DECIMAL(10,2)	Total amount due for the appointment.
PaidAmount	DECIMAL(10,2)	Amount the patient has paid so far.
RemainingBalance	DECIMAL(10,2)	Auto-calculated amount still owed.
ChangeAmount	DECIMAL(20,5)	Auto-calculated overpayment / change owed to the patient.
PaymentMethod	VARCHAR(50)	Cash / Card / Insurance.
PaymentStatus	VARCHAR(30)	Auto-calculated: Paid or Unpaid.
BillDate	DATE	Date the bill was issued.
PaymentDate	DATE	Date payment was completed (nullable until paid).
AppointmentID	INT, FK → APPOINTMENTS	The appointment this bill covers.

5.5 PRESCRIPTIONS

Medicine prescriptions issued during a specific appointment.

Column	Type	Purpose
PrescriptionID	INT, PK, AUTO_INCREMENT	Unique identifier for each prescription.
MedicineName	VARCHAR(100)	Name of the prescribed medicine.
Dosage_MgML	DECIMAL(10,2)	Dosage amount.
Frequency_PerDay	INT	Number of doses per day.
Duration_Weeks	INT	Prescription duration, in weeks.
AppointmentID	INT, FK → APPOINTMENTS	The appointment this prescription was issued during.

5.6 ROOMS

Physical hospital rooms and their live occupancy status.

Column	Type	Purpose
RoomID	INT, PK	Unique identifier for each room (manually assigned).
RoomNumber	VARCHAR(20)	Displayed room number.
FloorNumber	INT	Floor the room is located on.
AvailabilityStatus	VARCHAR(20)	Available or Occupied — kept in sync automatically by triggers.

5.7 Patient_Doctor_Consultation

Associative table modeling an ongoing many-to-many consultation relationship between doctors and patients, separate from individual appointments.

Column	Type	Purpose
DoctorID	INT, PK (composite), FK → DOCTORS	Consulting doctor.
PatientID	INT, PK (composite), FK → PATIENTS	Patient under consultation.

6. Triggers

Six triggers automate business rules directly at the database layer, ensuring calculated fields and dependent statuses stay consistent regardless of which application or user performs the write.

6.1 Insert_Bills — BEFORE INSERT ON Bills

Automatically calculates RemainingBalance, ChangeAmount, and PaymentStatus the moment a new bill is created, based on TotalAmount vs. PaidAmount.

```
DELIMITER //
CREATE TRIGGER Insert_Bills
BEFORE INSERT ON Bills
FOR EACH ROW
BEGIN
  IF NEW.TotalAmount > NEW.PaidAmount THEN
    SET NEW.RemainingBalance = NEW.TotalAmount - NEW.PaidAmount;
    SET NEW.ChangeAmount = 0;
    SET NEW.PaymentStatus = 'Unpaid';
  ELSE
    SET NEW.ChangeAmount = NEW.PaidAmount - NEW.TotalAmount;
    SET NEW.RemainingBalance = 0;
    SET NEW.PaymentStatus = 'Paid';
  END IF;
END //
DELIMITER ;
```

6.2 Update_Bills — BEFORE UPDATE ON Bills

Recalculates the same three fields whenever a bill is updated — most commonly when a patient makes a payment (PaidAmount changes).

```
DELIMITER //
CREATE TRIGGER Update_Bills
BEFORE UPDATE ON Bills
FOR EACH ROW
BEGIN
  IF OLD.TotalAmount > NEW.PaidAmount THEN
    SET NEW.RemainingBalance = OLD.TotalAmount - NEW.PaidAmount;
    SET NEW.ChangeAmount = 0;
    SET NEW.PaymentStatus = 'Unpaid';
  ELSE
    SET NEW.ChangeAmount = NEW.PaidAmount - OLD.TotalAmount;
    SET NEW.RemainingBalance = 0;
    SET NEW.PaymentStatus = 'Paid';
  END IF;
END //
DELIMITER ;
```

6.3 Update_doctors_salary — BEFORE UPDATE ON doctors

Recalculates a doctor's monthly Salary whenever their record is updated, using a fixed formula of 10 working hours/day and 20 working days/month.

```
DELIMITER //
CREATE TRIGGER Update_doctors_salary
BEFORE UPDATE ON doctors
FOR EACH ROW
BEGIN
    SET NEW.Salary = NEW.HourlyPay * 10 * 20;
END //
DELIMITER ;
```

6.4 Room_Availability_Insert — AFTER INSERT ON PATIENTS

Marks a room Occupied when a new patient is inserted actively using that room.

```
DELIMITER //
CREATE TRIGGER Room_Availability_Insert
AFTER INSERT ON PATIENTS
FOR EACH ROW
BEGIN
    IF NEW.Room_use = 1 THEN
        UPDATE ROOMS
        SET AvailabilityStatus = 'Occupied'
        WHERE RoomID = NEW.RoomID;
    END IF;
END //
DELIMITER ;
```

6.5 Room_Availability_update — AFTER UPDATE ON PATIENTS

Keeps ROOMS.AvailabilityStatus synced whenever a patient's Room_use flag transitions — freeing a room when a patient leaves, occupying it when a patient moves in.

```
DELIMITER //
CREATE TRIGGER Room_Availability_update
AFTER UPDATE ON PATIENTS
FOR EACH ROW
BEGIN
    IF OLD.Room_use = 1 AND NEW.Room_use = 0 THEN
        UPDATE ROOMS SET AvailabilityStatus = 'Available'
        WHERE RoomID = OLD.RoomID;
    ELSEIF OLD.Room_use = 0 AND NEW.Room_use = 1 THEN
        UPDATE ROOMS SET AvailabilityStatus = 'Occupied'
        WHERE RoomID = NEW.RoomID;
    END IF;
END //
DELIMITER ;
```

6.6 RoomID_Change — BEFORE UPDATE ON PATIENTS

Automatically clears a patient's RoomID when they stop using a room, and auto-assigns the first available room when they start using one.

```
DELIMITER //
CREATE TRIGGER RoomID_Change
BEFORE UPDATE ON PATIENTS
FOR EACH ROW
BEGIN
    IF OLD.Room_use = 1 AND NEW.Room_use = 0 THEN
        SET NEW.RoomID = NULL;
    ELSEIF OLD.Room_use = 0 AND NEW.Room_use = 1 THEN
        SET NEW.RoomID = (
            SELECT RoomID FROM ROOMS
            WHERE AvailabilityStatus = 'Available'
            LIMIT 1
        );
    END IF;
END //
DELIMITER ;
```

7. Views

Four views simplify the most common read patterns into single, join-ready queries.

7.1 Doctor_Schedule_View

Shows each doctor's upcoming or scheduled appointments with readable patient names instead of raw IDs.

```
CREATE VIEW Doctor_Schedule_View AS
SELECT d.FullName AS DoctorName, p.FullName AS PatientName,
       a.AppointmentDate, a.AppointmentStatus, a.Diagnosis
FROM APPOINTMENTS a
JOIN PATIENTS p ON a.PatientID = p.PatientID
JOIN DOCTORS d ON a.DoctorID = d.DoctorID
WHERE a.AppointmentDate >= NOW() OR a.AppointmentStatus = 'Scheduled';
```

7.2 Patient_Bill_Summary_View

Aggregates each patient's total billed, paid, and outstanding amounts across all of their appointments.

```
CREATE VIEW Patient_Bill_Summary_View AS
SELECT p.PatientID, p.FullName AS PatientName,
       SUM(b.TotalAmount) AS TotalBilled,
       SUM(b.PaidAmount) AS TotalPaid,
       SUM(b.RemainingBalance) AS TotalOwed
FROM PATIENTS p
JOIN APPOINTMENTS a ON p.PatientID = a.PatientID
JOIN BILLS b ON a.AppointmentID = b.AppointmentID
GROUP BY p.PatientID, p.FullName;
```

7.3 Room_Occupancy_View

Shows the live occupancy of every room, including which patient (if any) currently occupies it.

```
CREATE VIEW Room_Occupancy_View AS
SELECT r.RoomID, r.RoomNumber, r.FloorNumber, r.AvailabilityStatus,
       p.PatientID, p.FullName AS PatientName
FROM ROOMS r
LEFT JOIN PATIENTS p ON r.RoomID = p.RoomID;
```

7.4 Doctor_Earnings_View

Lists doctors with their calculated salary, grouped by specialty for reporting.

```
CREATE VIEW Doctor_Earnings_View AS
SELECT FullName AS DoctorName, Speciality, Salary
FROM DOCTORS
ORDER BY Speciality, Salary DESC;
```

8. Stored Procedures

Five stored procedures encapsulate the core operational actions a clinic application would perform, several of which trigger the automation described in Section 6.

8.1 AddNewAppointment

Creates a new Scheduled appointment for a given patient and doctor.

```
CREATE PROCEDURE AddNewAppointment(
  IN p_PatientID INT, IN p_DoctorID INT, IN p_Diagnosis VARCHAR(255)
)
BEGIN
  INSERT INTO APPOINTMENTS
    (AppointmentStatus, Diagnosis, AppointmentDate, PatientID, DoctorID)
  VALUES ('Scheduled', p_Diagnosis, NOW(), p_PatientID, p_DoctorID);
END
```

8.2 DischargePatient

Discharges a patient by setting Room_use = 0 — which automatically cascades through Room_Availability_update and RoomID_Change to free the room.

```
CREATE PROCEDURE DischargePatient(IN p_PatientID INT)
BEGIN
  UPDATE PATIENTS SET Room_use = 0 WHERE PatientID = p_PatientID;
END
```

8.3 GetPatientHistory

Returns a complete patient record in one call: personal info, appointment history, prescriptions, and bills — four result sets from a single stored procedure.

```
CREATE PROCEDURE GetPatientHistory(IN p_PatientID INT)
BEGIN
  SELECT PatientID, FullName, Phone, Email, BloodType, RoomID, Room_use
  FROM PATIENTS WHERE PatientID = p_PatientID;

  SELECT AppointmentID, AppointmentStatus, Diagnosis, AppointmentDate, DoctorID
  FROM APPOINTMENTS WHERE PatientID = p_PatientID;

  SELECT pr.PrescriptionID, pr.MedicineName, pr.Dosage_MgML,
    pr.Frequency_PerDay, pr.Duration_Weeks, pr.AppointmentID
  FROM PRESCRIPTIONS pr
  JOIN APPOINTMENTS a ON pr.AppointmentID = a.AppointmentID
  WHERE a.PatientID = p_PatientID;

  SELECT b.BillID, b.TotalAmount, b.PaidAmount, b.RemainingBalance,
    b.ChangeAmount, b.PaymentMethod, b.PaymentStatus,
    b.BillDate, b.PaymentDate, b.AppointmentID
  FROM BILLS b
  JOIN APPOINTMENTS a ON b.AppointmentID = a.AppointmentID
  WHERE a.PatientID = p_PatientID;
END
```


8.4 ProcessPayment

Adds a payment amount to a bill's PaidAmount and stamps the payment date — the Update_Bills trigger then automatically recalculates balance, change, and payment status.

```
CREATE PROCEDURE ProcessPayment(IN p_BillID INT, IN p_AmountPaid DECIMAL(10,2))
BEGIN
  UPDATE BILLS
  SET PaidAmount = PaidAmount + p_AmountPaid, PaymentDate = NOW()
  WHERE BillID = p_BillID;
END
```

8.5 AssignDoctorToPatient

Creates a Patient–Doctor Consultation link, representing an ongoing consultation relationship independent of any single appointment.

```
CREATE PROCEDURE AssignDoctorToPatient(IN p_DoctorID INT, IN p_PatientID INT)
BEGIN
  INSERT INTO Patient_Doctor_Consultation (DoctorID, PatientID)
  VALUES (p_DoctorID, p_PatientID);
END
```

9. Indexing

Three indexes were added on the foreign key columns most frequently used in joins and filters, reducing lookup cost as the dataset grows.

Index	Column	Purpose
idx_appointments_patientID	APPOINTMENTS(PatientID)	Speeds up filtering/joining appointments by patient.
idx_appointments_doctorID	APPOINTMENTS(DoctorID)	Speeds up filtering/joining appointments by doctor.
idx_bills_appointmentID	BILLS(AppointmentID)	Speeds up joining bills back to their appointment.

10. Subqueries

Eight subqueries demonstrate common filtering and aggregation patterns layered on top of the schema — membership checks (IN), scalar comparisons against an aggregate, and a correlated subquery — covering reporting needs that a single flat query can't express cleanly.

10.1 Patients with at least one completed appointment

Uses IN with a subquery over APPOINTMENTS to find patients who have completed at least one visit.

```
SELECT PatientID, FullName
FROM PATIENTS
WHERE PatientID IN (
    SELECT DISTINCT PatientID
    FROM APPOINTMENTS
    WHERE AppointmentStatus = 'Completed'
);
```

10.2 Doctors earning above the clinic's average hourly pay

A scalar subquery calculates the average HourlyPay across all doctors, then filters against it.

```
SELECT DoctorID, FullName, Speciality, HourlyPay
FROM DOCTORS
WHERE HourlyPay > (
    SELECT AVG(HourlyPay)
    FROM DOCTORS
);
```

10.3 Bills above the average total amount

Same scalar-subquery pattern applied to BILLS, surfacing above-average charges.

```
SELECT BillID, TotalAmount, PaymentStatus, AppointmentID
FROM BILLS
WHERE TotalAmount > (
    SELECT AVG(TotalAmount)
    FROM BILLS
);
```

10.4 Scheduled-appointment count per patient

A correlated subquery in the SELECT list counts each patient's own Scheduled appointments row by row.

```
SELECT FullName,
    (SELECT COUNT(*)
     FROM APPOINTMENTS a
     WHERE a.PatientID = p.PatientID
     AND a.AppointmentStatus = 'Scheduled') AS ScheduledAppointments
FROM PATIENTS p;
```

10.5 Patients never assigned to a room

Nested NOT IN / IN subqueries identify patients with no room usage on record.

```
SELECT PatientID, FullName, Phone
FROM PATIENTS
WHERE PatientID NOT IN (
    SELECT DISTINCT PatientID
    FROM APPOINTMENTS
    WHERE PatientID IN (
        SELECT PatientID
        FROM PATIENTS
        WHERE RoomID IS NOT NULL
    )
);
```

10.6 Highest-paid doctor

A scalar MAX() subquery pinpoints the top earner in DOCTORS.

```
SELECT DoctorID, FullName, Speciality, Salary
FROM DOCTORS
WHERE Salary = (
    SELECT MAX(Salary)
    FROM DOCTORS
);
```

10.7 Appointments with an unpaid bill

Uses IN against BILLS to surface appointments still carrying an unpaid balance.

```
SELECT AppointmentID, Diagnosis, AppointmentDate, AppointmentStatus
FROM APPOINTMENTS
WHERE AppointmentID IN (
    SELECT AppointmentID
    FROM BILLS
    WHERE PaymentStatus = 'UnPaid'
);
```

10.8 Patients prescribed more than one medication

Joins APPOINTMENTS to PRESCRIPTIONS, then uses GROUP BY / HAVING inside the subquery to keep only patients with multiple prescriptions.

```
SELECT PatientID, FullName
FROM PATIENTS
WHERE PatientID IN (
    SELECT a.PatientID
    FROM APPOINTMENTS a
    JOIN PRESCRIPTIONS pr ON a.AppointmentID = pr.AppointmentID
    GROUP BY a.PatientID
    HAVING COUNT(pr.PrescriptionID) > 1
);
```

11. Project Summary

SmartClinic demonstrates a complete relational database lifecycle: conceptual modeling (ERD), logical-to-physical translation (Relational Mapping), requirements modeling from a user perspective (Use Case Diagram), and a fully implemented MySQL schema with seven normalized tables, six business-rule-enforcing triggers, four reporting views, five encapsulated stored procedures, targeted indexing for performance, and eight subqueries covering common filtering and aggregation patterns.

Skills Demonstrated

- Entity-Relationship modeling and relational schema design.
- Normalization and foreign key constraint design (including cascade/set-null delete behavior).
- Procedural SQL: triggers (BEFORE/AFTER, INSERT/UPDATE, OLD vs. NEW row semantics).
- Views for data abstraction and simplified reporting.
- Stored procedures for encapsulating multi-step operational logic.
- Performance-oriented indexing on high-traffic foreign key columns.
- Subquery-based filtering and aggregation: IN/NOT IN membership checks, scalar comparisons against an aggregate, and a correlated subquery.
- Use case / requirements modeling across multiple system actors.